SERVER SCRIPT

SHIFT PRODUCTION ENTRY

Script Type: API

Event Frequency: All

DocType Event: Before Insert

API Method: shift_production_entry.get_shift_details

Script

```
# Script: shift_production_entry (Proxy for get_shift_details)
# Logic: Fetches production and consumption details for a specific shift

posting_date = frappe.form_dict.get('posting_date')
shift = frappe.form_dict.get('shift')
unit = frappe.form_dict.get('unit')

if not posting_date or not shift:
    frappe.throw("Posting Date and Shift are required")

# 1. Fetch matching Work Orders from Roll Production Entry
rpe_filters = {
    "docstatus": 1,
    "posting_date": posting_date,
    "shift": shift,
    "unit": unit
}

rpe_docs = frappe.get_all("Roll Production Entry",
    filters=rpe_filters,
    fields=["work_order", "shift_batch_number"]
)

valid_work_orders = []
base_batch_no = ""
for rpe in rpe_docs:
    wo_name = rpe.get("work_order")
    if wo_name:
        valid_work_orders.append(wo_name)
    if not base_batch_no and rpe.get("shift_batch_number"):
        base_batch_no = rpe.get("shift_batch_number")

if not valid_work_orders:
    frappe.response['message'] = {"production_items": [], "consumption_items": [], "base_batch_no": ""}
    frappe.msgprint(f"No submitted Roll Production Entries found for {unit} on {posting_date} ({shift}).")
else:
    # 2. Fetch Stock Entries
    stock_entries = frappe.get_all("Stock Entry"
```

```
    stock_entries = frappe.get_all("Stock Entry",
        fields=["name", "work_order", "from_warehouse", "to_warehouse", "bom_no"],
        filters={"docstatus": 1, "purpose": "Manufacture", "work_order": ["in", valid_work_orders]}
    )

    production_map = {}
    consumption_map = {}

    for se in stock_entries:
        fg_items = frappe.get_all("Stock Entry Detail",
            filters={"parent": se.name, "is_finished_item": 1},
            fields=["item_code", "item_name", "qty", "uom"]
        )

        for fg in fg_items:
            key = (fg.item_code, se.work_order, se.to_warehouse)
            if key not in production_map:
                production_map[key] = {
                    "work_order": se.work_order, "stock_entry": se.name, "item_code": fg.item_code,
                    "item_name": fg.item_name, "produced_qty": 0.0, "fg_warehouse": se.to_warehouse
                }
            else:
                curr_se = str(production_map[key]["stock_entry"])
                if se.name not in curr_se: production_map[key]["stock_entry"] = curr_se + ", " + se.name

            p_data = production_map[key]
            p_data["produced_qty"] = p_data["produced_qty"] + float(fg.qty or 0)

            bom_no = se.bom_no or frappe.db.get_value("Work Order", se.work_order, "bom_no")
            if bom_no:
                bom_qty = float(frappe.db.get_value("BOM", bom_no, "quantity") or 1) or 1
                bom_items = frappe.get_all("BOM Item", filters={"parent": bom_no}, fields=["item_code", "item_name", "q
ty", "stock_uom"])

                for rm in bom_items:
                    req_qty = (float(rm.qty) / bom_qty) * float(fg.qty)
                    if rm.item_code not in consumption_map:
                        consumption_map[rm.item_code] = {
                            "item_code": rm.item_code, "item_name": rm.item_name, "uom": rm.stock_uom,
                            "standard_consumption": 0.0, "actual_consumption": 0.0
                        }
                    consumption_map[rm.item_code]["standard_consumption"] = consumption_map[rm.item_code]["stand
ard_consumption"] + req_qty
                    consumption_map[rm.item_code]["actual_consumption"] = consumption_map[rm.item_code]["actual_c
onsumption"] + req_qty

    frappe.response['message'] = {
```

```
    "production_items": list(production_map.values()),
    "consumption_items": list(consumption_map.values()),
    "base_batch_no": base_batch_no
}
```

Request Limit:	5
Time Window (Seconds):	86400